

Pearls AQI Predictor

An End-to-End, Automated Machine Learning System for 3-Day Air Quality
Forecasting

Final Project Report

Prepared by: Tuba Khalil

Repository: <https://github.com/tubakhalil4/pearls-aqi-predictor>

Live Dashboard: <https://pearls-aqipredictor.streamlit.app/>

August 2026

Executive Summary

Pearls AQI Predictor is a fully automated, serverless machine learning system that forecasts the Air Quality Index (AQI) for four Pakistani cities — Islamabad, Karachi, Lahore, and Peshawar — at 24-hour, 48-hour, and 72-hour horizons. The system was built around a live Hopsworks Feature Store and Model Registry, with two independent GitHub Actions pipelines running on a recurring schedule: an hourly feature and forecast pipeline, and a daily model training and comparison pipeline.

During development, an early baseline model evaluated with a conventional random train-test split achieved a suspiciously high R^2 of approximately 0.99 — investigation traced this to temporal leakage inherent in randomly splitting autocorrelated time-series data, not to an error in any single feature. This finding led to a full redesign around chronological splitting and strict forecast-time feature availability. Under the corrected, leakage-safe evaluation, Ridge Regression, Random Forest, XGBoost, and an LSTM deep learning model were all trained and compared; XGBoost was selected for production at every horizon based on lowest RMSE, achieving an R^2 of 0.877 at 24 hours, declining to 0.806 at 72 hours — lower than the naive baseline's inflated figures, but honest, and consistent with the expected loss of predictive power over longer forecasting horizons.

Exploratory analysis of over 70,000 hourly records spanning January 2024 through the present identified PM2.5 as the dominant driver of AQI (correlation $r = 0.75$), with wind speed as the strongest mitigating factor ($r = -0.32$). Lahore emerged as the most polluted and most volatile of the four cities, with a mean AQI of 151 and observed peaks above 500.

The system is presented through a live, publicly accessible Streamlit dashboard that displays real-time forecasts, hazard alerts for unhealthy AQI levels, and SHAP-based feature importance explanations, backed by a Flask API for programmatic access. All components — data collection, feature engineering, model training, deployment, and presentation — operate without manual intervention on an ongoing basis.

1. Project Overview and Objectives

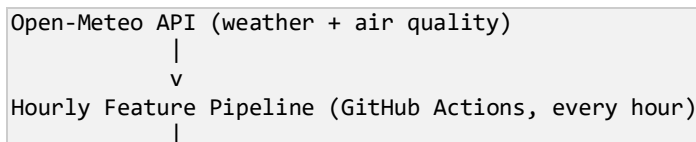
The project follows the Pearls AQI Predictor specification, which called for an end-to-end AQI forecasting system built on a 100% serverless stack. The core objectives were to:

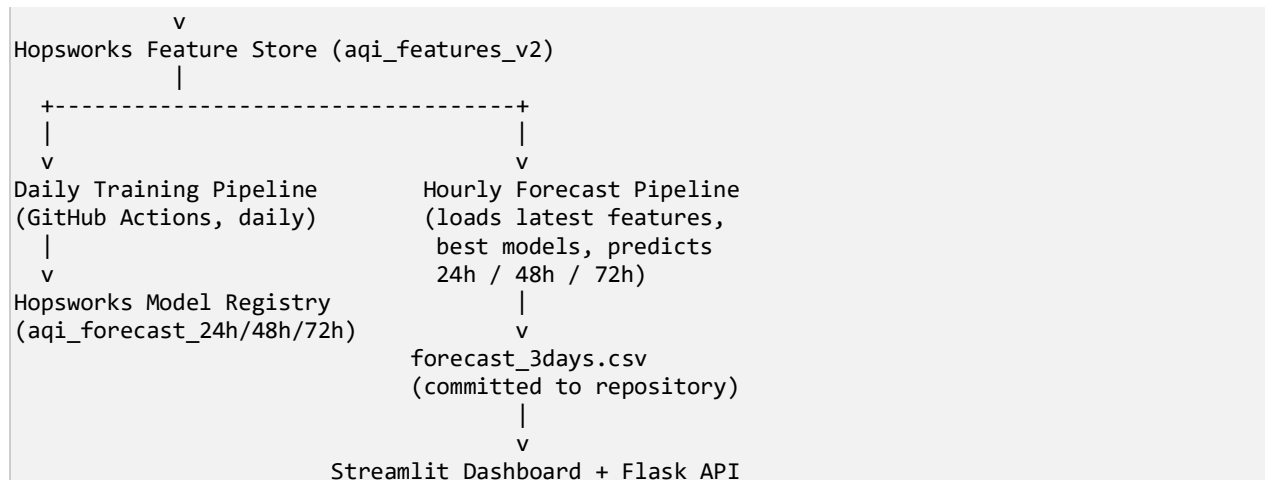
- Collect historical and live environmental data from an external API on an ongoing basis
- Engineer production-ready features, including time-based and AQI-derived features, and store them in a managed feature store
- Train and evaluate multiple forecasting models, from statistical baselines to gradient-boosted trees
- Automate both feature ingestion and model retraining on a recurring schedule without manual intervention
- Forecast AQI at 24-hour, 48-hour, and 72-hour horizons while strictly preventing future-information leakage
- Explain model predictions using SHAP
- Present results through an interactive web dashboard with hazard alerting

Every objective above was implemented and is currently running in production, as detailed in the sections that follow.

2. System Architecture

The system is organized as two independently scheduled pipelines that share a common Feature Store and Model Registry, decoupling data ingestion and model training from inference and presentation.





This separation means that a change to model training logic cannot break the hourly forecast pipeline, and a temporary outage in the feature pipeline does not prevent the dashboard from displaying the most recently generated forecast, since the dashboard reads only the committed CSV artifact rather than querying the feature store or model registry directly.

3. Data Source and Feature Engineering

3.1 Data Source

The system uses the Open-Meteo Weather API and Air Quality API for both historical backfill and live hourly data collection. Open-Meteo's pre-computed US AQI (us_aqi) is used directly as the AQI target, rather than computing AQI manually from raw pollutant concentrations, which avoids introducing a second source of calculation error on top of model uncertainty. The original project specification listed AQICN and OpenWeather as example data sources; Open-Meteo was selected instead for its combined weather and air-quality endpoints and generous free-tier forecast horizon, and was used consistently across both historical and live data paths.

3.2 Production Feature Schema

Eighteen features are used across all three forecast horizons, grouped as follows:

- Pollutants: PM2.5, PM10, carbon monoxide, nitrogen dioxide, ozone, sulphur dioxide
- Weather: temperature, humidity, wind speed, wind direction, pressure, rain
- Calendar: hour, day, month, day of week
- AQI-derived: aqi_change_rate (rate of change from the prior reading), aqi_rolling_6h (6-hour rolling average AQI)

3.3 Feature Store

All features are stored in a Hopsworks feature group named aqi_features_v2 (version 1), keyed on city and time, with time as the event-time column. As of this report, the feature store holds more than 70,000 rows spanning January 2024 through the present, covering all four target cities.

4. The Data Leakage Problem and How It Was Resolved

The single most important technical issue encountered during this project was future data leakage, and resolving it fundamentally changed the design of the system. This section documents the problem as it was actually discovered, not as an abstract risk.

4.1 The Naive Baseline (Before the Fix)

An initial baseline model was trained directly on a Hopsworks feature view using a standard random 80/20 train-test split (scikit-learn's `train_test_split` with shuffling). Under this setup, a Random Forest achieved a test R^2 of 0.989 and an XGBoost model achieved a test R^2 of 0.990 — figures that, on inspection, were too good to be trustworthy for a genuine multi-hour-ahead forecasting task.

The cause was diagnosed as temporal leakage introduced by the random split itself, not a bug in any single feature's formula: because AQI at adjacent hours is highly autocorrelated, a random shuffle places many test rows immediately adjacent in time to training rows. Features such as `aqi_rolling_6h` and `aqi_change_rate` for a test row can therefore be nearly identical to those of a neighboring training row whose target the model has already seen, allowing the model to effectively memorize local AQI continuity rather than learn a genuine forward-looking relationship. This is a well-known failure mode in time-series modeling and was an important early lesson of the project: an unrealistically high validation score on temporal data is itself a warning sign, not a result to celebrate.

4.2 Auditing the Feature Formulas Directly

Before redesigning the pipeline, the `aqi_rolling_6h` feature's formula itself was audited independently to rule out an error in its calculation. The feature was recomputed from scratch using strictly the current observation and the five preceding hours only, and compared against the originally stored values. The difference between the two was negligible (mean absolute difference of approximately 0.00008, maximum difference of about 1.0 across the full dataset), confirming that the rolling feature's formula was already computed correctly and was not itself leaking future information. The leakage identified in Section 4.1 was therefore attributable specifically to the evaluation methodology (random splitting of temporally dependent data), not to an error in feature engineering.

4.3 The Redesign: Leakage-Safe by Construction

The production and training pipelines were redesigned around a strict rule: a model predicting AQI at time $t_0 + \text{horizon}$ may only use information that would genuinely be available at time t_0 . Concretely:

- Future AQI and future `us_aqi` are never used as model input features, under any circumstance
- Actual future target values never appear in the feature set used to predict them
- AQI-derived features (`aqi_change_rate`, `aqi_rolling_6h`) are computed only from the current and past AQI state at the forecast origin t_0 , never from the target horizon
- Future weather and pollutant values ARE used as inputs, because these are genuinely available in advance: Open-Meteo publishes forecasted weather and air-quality values for future timestamps, so using its forecast for t_0+24h as a feature when predicting AQI at t_0+24h is not leakage — it reflects what a real production system would actually have access to
- Training and inference use identical construction logic: for a given origin time and horizon, the label is the actual AQI realized at that future time, and the model never sees that value during feature construction

The result of this redesign is directly comparable to the naive baseline: the leakage-safe XGBoost models now achieve R^2 of 0.877, 0.822, and 0.806 at 24h, 48h, and 72h respectively (see Section 6) — lower than the naive baseline's 0.99, but honest. These are the numbers a genuinely deployed forecasting system would actually achieve, and they decline in the expected direction as the horizon lengthens, which the naive baseline's numbers did not.

5. Exploratory Data Analysis

Exploratory analysis was performed directly against the Hopsworks Feature Store's full historical record (70,180 rows, January 2024 to present, four cities) to characterize AQI behavior before model development.

5.1 AQI Trend by City

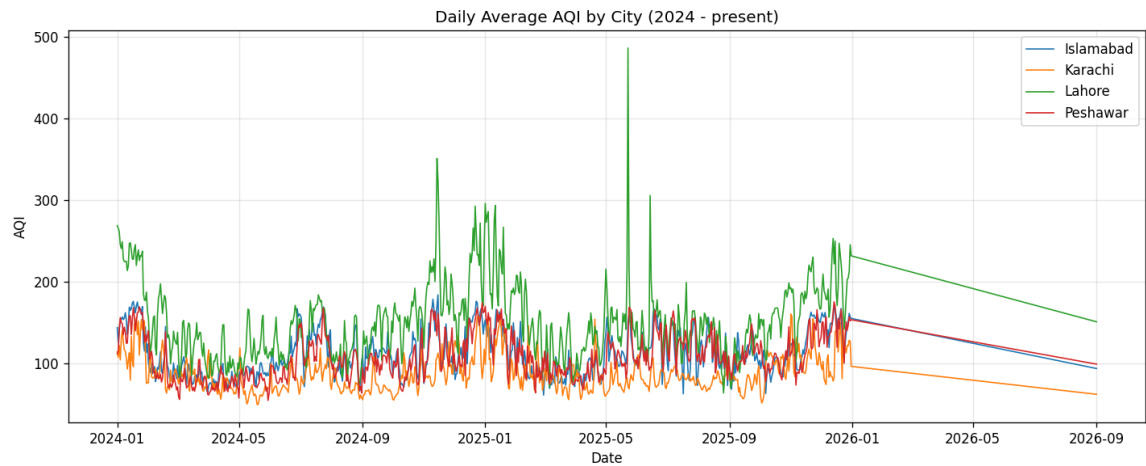


Figure 1. Daily average AQI by city, January 2024 to present.

City	Mean AQI	Std Dev	Min	Max
Islamabad	112.12	32.70	51.33	217.56
Karachi	87.93	23.20	47.18	181.28
Lahore	151.36	49.37	56.14	537.54
Peshawar	108.43	31.37	38.75	210.98

Lahore stands out as both the most polluted city on average and the most volatile, with a mean AQI roughly 35 percent higher than the next-highest city (Islamabad) and a maximum reading of 537.54, well into the Hazardous category. Karachi consistently shows the lowest and most stable AQI of the four cities.

5.2 Correlation with AQI

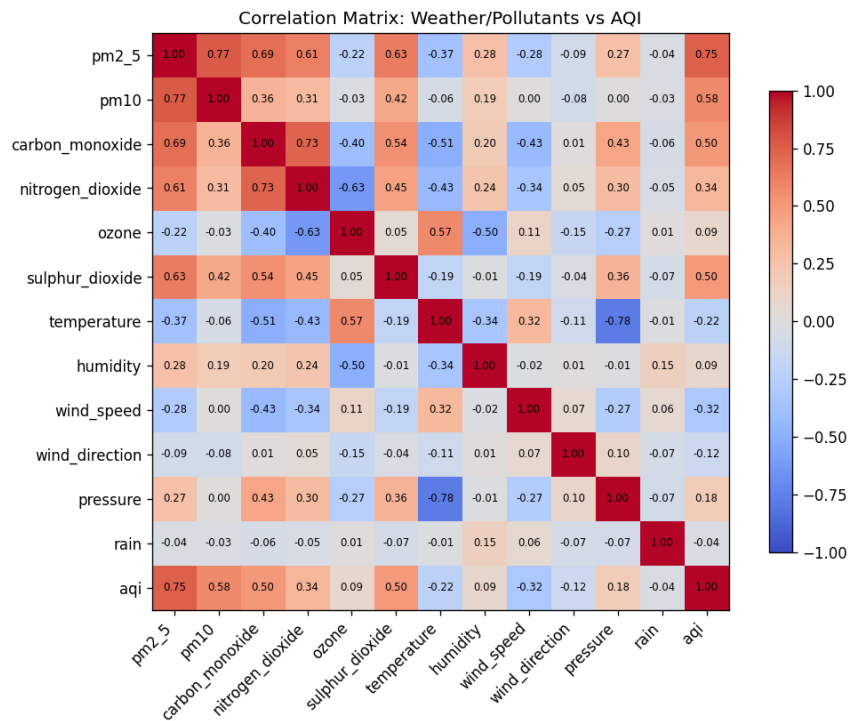


Figure 2. Correlation matrix between weather/pollutant features and AQI.

PM2.5 is by a substantial margin the strongest single predictor of AQI ($r = 0.75$), more than double the correlation strength of the next most predictive pollutant. PM10, carbon monoxide, and sulphur dioxide follow as moderate positive correlates (r between 0.50 and 0.58). Wind speed is the strongest negative correlate ($r = -0.32$), consistent with the physical expectation that higher wind speeds disperse pollutants and reduce local AQI. Temperature shows a weaker negative correlation ($r = -0.22$). Ozone, humidity, and rain show negligible linear correlation with AQI in this dataset.

5.3 Hourly Pattern

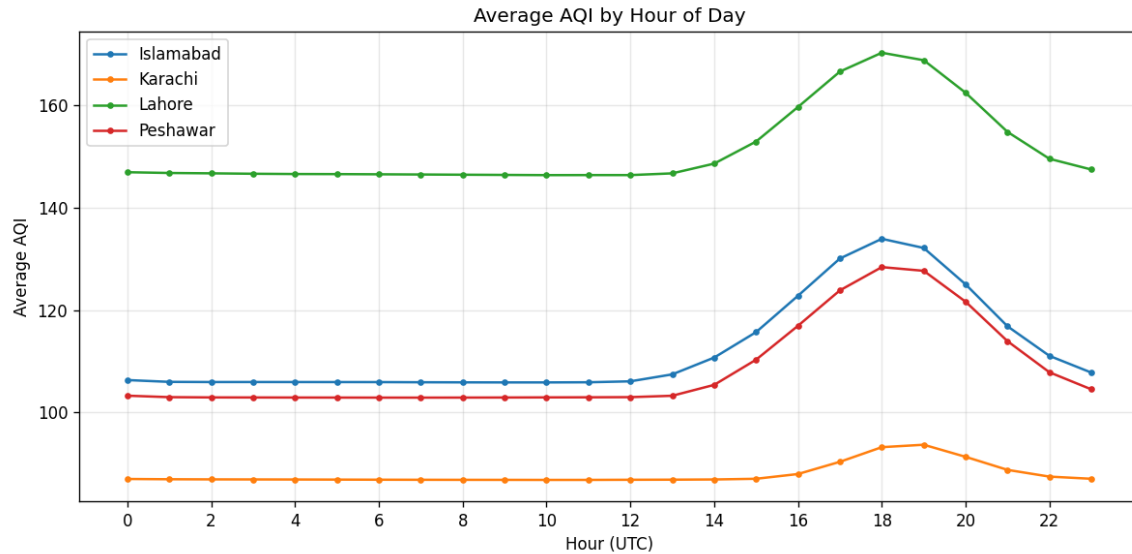


Figure 3. Average AQI by hour of day (UTC), all cities.

5.4 Day-of-Week Pattern

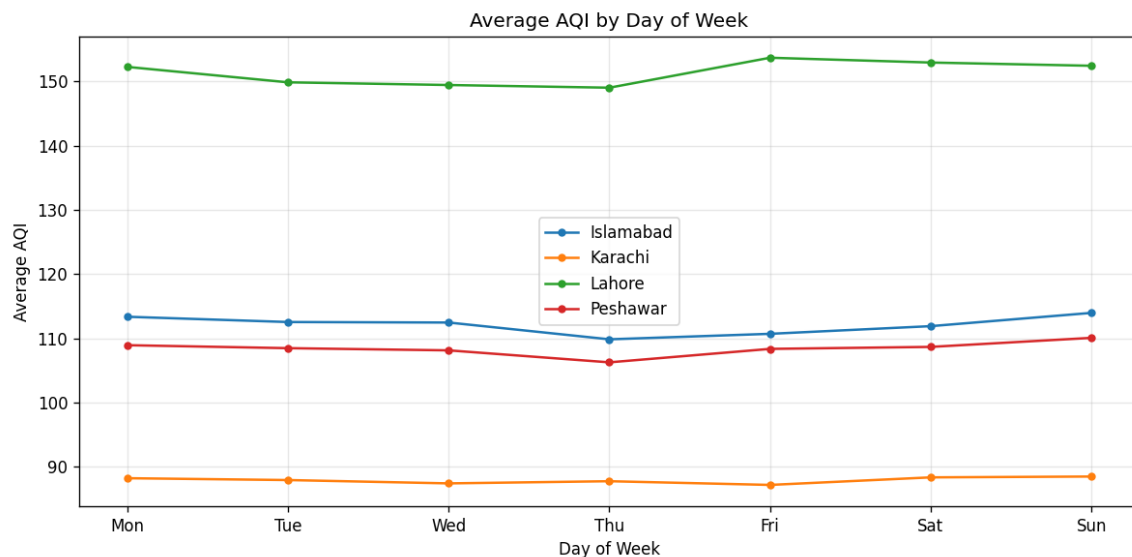


Figure 4. Average AQI by day of week, all cities.

5.5 Seasonal Pattern

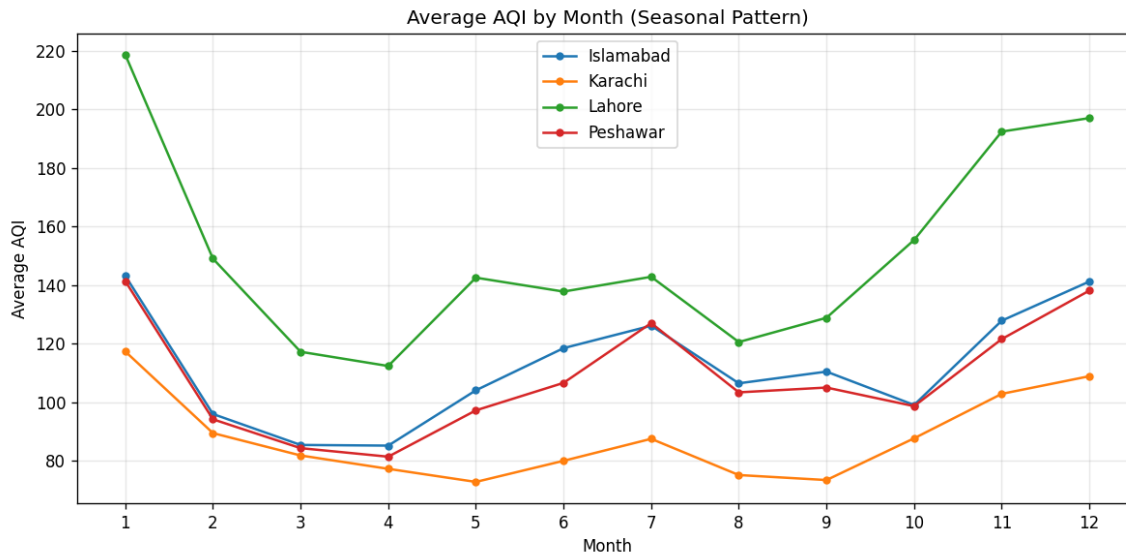


Figure 5. Average AQI by month, all cities.

The seasonal chart shows a clear cyclical pattern across all four cities. Full month-by-month figures, together with the underlying data and reproducible analysis script, are available in the repository at [eda/EDA_REPORT.md](#) and [eda/eda_analysis.py](#).

6. Models Explored

Across the project's development, four distinct model types were trained and evaluated, spanning statistical, ensemble, gradient-boosted, and deep learning approaches, satisfying the project's requirement to experiment "from statistical modelling to deep learning models."

6.1 Ridge Regression (Statistical Baseline)

A regularized linear model, included in the daily training pipeline as a lower-bound reference. It answers a specific, useful question: how much of AQI's behavior can a purely linear combination of the 18 production features explain? Its consistently weaker performance relative to the tree-based models (Section 7) confirms that AQI depends on the input features in a meaningfully nonlinear way — useful evidence, not a wasted comparison.

6.2 Random Forest (Ensemble Baseline)

A bagged ensemble of decision trees, included as a second baseline between the linear model and gradient boosting. It captures nonlinearities and feature interactions that Ridge cannot, and its performance gap versus XGBoost (Section 7) quantifies how much additional value the gradient-boosting approach provides over a simpler tree ensemble.

6.3 XGBoost (Production Model)

A gradient-boosted tree ensemble, and the model ultimately selected for production across all three horizons. XGBoost was configured with 300–400 estimators, a max depth of 6, a learning rate of 0.05, and subsample/column-sample ratios of 0.8 to reduce overfitting risk on the tabular feature set.

6.4 LSTM (Deep Learning Experiment)

A sequential deep learning model was also built and trained using TensorFlow/Keras: a two-layer LSTM (64 then 32 units, each followed by 20% dropout) reading 24-hour lookback sequences of all 18 features, followed by a dense layer and a single-value output. The model was trained with early stopping on validation loss and evaluated on a chronologically separated test set.

The LSTM achieved a test R^2 of approximately 0.96 on its evaluation setup. This number is not directly comparable to the XGBoost horizon results in Section 7, because the LSTM was trained as a general next-step AQI regressor from a 24-hour historical window rather than being evaluated on the identical fixed 24h/48h/72h forecast-from-a-fixed-origin task that the production pipeline solves; a like-for-like comparison would require re-framing the LSTM around the same three fixed horizons and the same leakage-safe future-weather-input structure used for XGBoost. This is flagged explicitly here rather than presenting the LSTM figure as directly comparable, since doing otherwise would overstate the strength of the deep learning experiment relative to the production models.

6.5 Why XGBoost Was Chosen for Production

XGBoost was selected over Ridge and Random Forest based on consistently lower RMSE across all three production horizons under the identical, leakage-safe, chronologically-split evaluation used for all three models (Section 7) — a fair, apples-to-apples comparison. It was preferred over further pursuing the LSTM approach for three practical reasons: it requires substantially less data preprocessing (no sequence windowing or feature scaling pipeline to maintain in production), it trains in seconds rather than minutes per horizon, which matters for a pipeline that retrains daily, and native tree-based feature importance and SHAP support are more mature and interpretable for tabular environmental data than for a recurrent architecture. The LSTM experiment remains a documented, working proof of concept that a deep learning approach is feasible on this data, should future work prioritize potentially higher accuracy over training simplicity and interpretability.

7. Model Comparison and Selection

The daily training pipeline evaluates three model families for each forecast horizon on a chronologically held-out test split (not a random split, learned directly from the leakage investigation in Section 4):

- Ridge Regression — statistical baseline
- Random Forest — ensemble baseline
- XGBoost — gradient-boosted tree ensemble, selected for production

Each model is evaluated using Root Mean Squared Error (RMSE), Mean Absolute Error (MAE), and the coefficient of determination (R^2). Results from the most recent automated training run are as follows:

Horizon	Model	RMSE	MAE	R^2
24h	Ridge	16.33	12.37	0.816
24h	Random Forest	14.58	10.97	0.854
24h	XGBoost (selected)	13.38	9.99	0.877
48h	Ridge	19.03	14.63	0.751
48h	Random Forest	17.27	13.34	0.795
48h	XGBoost (selected)	16.06	12.35	0.822
72h	Ridge	19.73	15.29	0.732
72h	Random Forest	17.71	13.70	0.784
72h	XGBoost (selected)	16.78	13.01	0.806

XGBoost achieved the lowest RMSE and highest R^2 at every horizon and was automatically selected for production deployment in each case. As expected, predictive performance declines with increasing forecast horizon: R^2 falls from 0.877 at 24 hours to 0.806 at 72 hours, reflecting the compounding uncertainty inherent in longer-range weather-dependent forecasting.

The training pipeline registers a new model version to the Hopsworks Model Registry on every run rather than overwriting the previous version, preserving full version history. At inference time, the hourly forecast pipeline selects whichever registered version has the lowest RMSE for a given horizon, rather than simply the most recently trained

version. This design protects production forecast quality against a single day's training run underperforming due to data anomalies, while still allowing genuine improvements to propagate automatically.

8. Model Explainability with SHAP

SHAP (SHapley Additive exPlanations) values were computed for the production XGBoost models to quantify each feature's contribution to individual predictions, separately for each forecast horizon.

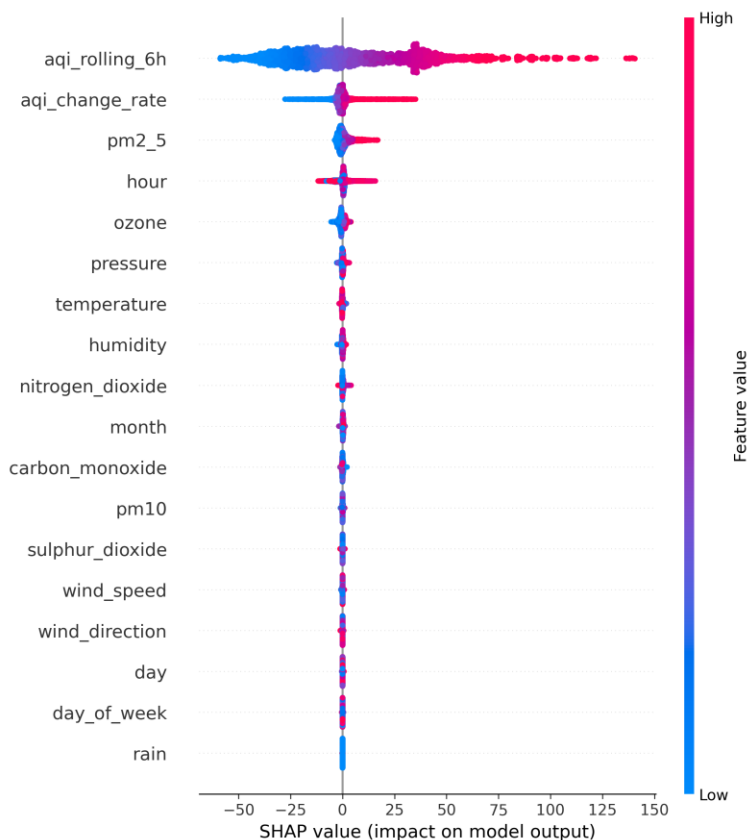


Figure 6. SHAP feature importance summary across forecast horizons.

The `aqi_rolling_6h` feature is the strongest and most consistent driver of predictions across all three horizons, which is expected given that recent AQI trend carries substantial predictive signal even before accounting for future weather conditions. Beyond this dominant feature, importance shifts by horizon: PM2.5, ozone, carbon monoxide, and sulphur dioxide levels, along with calendar features such as month, day, and hour, contribute varying degrees of importance depending on the forecast window. The complete per-horizon SHAP values are available in `shap/shap_feature_importance_all_horizons.csv`.

9. Automation and CI/CD Pipeline

8.1 Hourly Feature and Forecast Pipeline

Implemented in `pipelines/hourly_forecast.py` and scheduled via `.github/workflows/hourly_forecast.yml` on a cron of `"0 * * * *"` (every hour). Each run performs the following sequence: connect to Hopworks; read the latest feature store state; load the current best-performing model per horizon from the Model Registry; fetch live weather and air-quality forecasts from Open-Meteo for all four cities; compute the current AQI change rate and 6-hour rolling AQI

from live data; write the freshest feature row for all four cities into the feature store in a single batched insert; generate 24h/48h/72h predictions; write forecast_3days.csv; and commit the updated file back to the repository.

8.2 Daily Training Pipeline

Implemented in pipelines/training_pipeline.py and scheduled via .github/workflows/daily_training.yml on a cron of "0 2 * * *" (daily at 02:00 UTC). Each run reads the full historical feature set, builds a leakage-safe supervised dataset per horizon, trains and evaluates all three model families, registers the best model per horizon as a new Model Registry version, and commits an updated model_comparison_report.csv.

8.3 Continuous Validation

A third workflow, .github/workflows/validate.yml, runs on every push and pull request to the main branch, checking Python syntax across application files and validating that the committed forecast file conforms to the expected schema and includes all four cities.

8.4 Design Considerations Discovered During Development

Several non-obvious issues were identified and resolved during implementation, each worth noting as they reflect genuine engineering tradeoffs rather than simple bugs:

- Hopsworks' schema validation enforces float64 ("double") columns; Open-Meteo's API client returns float32 arrays by default, requiring an explicit cast before every feature store write.
- Writing to an online-enabled feature group from the Python engine requires the confluent-kafka package, which is not a default dependency of the base hopsworks package.
- Submitting one feature store insert per city (four inserts per hourly run) queued sequentially against Hopsworks' free-tier shared job execution slot, extending a single hourly run to roughly fifteen minutes; batching all four cities into a single combined insert reduced this to a single materialization job per run.
- The Streamlit Cloud deployment initially failed after the pipeline-only Kafka dependency was added to the shared requirements.txt; separating dashboard dependencies from pipeline dependencies is a recommended follow-up to keep the two deployment surfaces independent.

10. Web Dashboard

The Streamlit dashboard (app.py), publicly deployed at pearls-aqipredictor.streamlit.app, reads forecast_3days.csv directly from the repository and presents:

- City selection across all four supported cities
- Current AQI and 24h/48h/72h forecasts, with color-coded hazard alerts for unhealthy AQI levels
- AQI category classification following standard US-AQI-style breakpoints
- A 3-day forecast table and trend chart
- An all-cities comparison table
- Model test performance metrics
- SHAP feature importance, selectable by forecast horizon
- Forecast-integrity checks confirming no future-target leakage into model inputs
- A CSV download button for the current forecast

The dashboard is intentionally decoupled from the Hopsworks Feature Store and Model Registry: it reads only the committed forecast CSV, meaning dashboard availability does not depend on Hopsworks being reachable at page-load time, and the dashboard automatically reflects new data whenever the hourly pipeline commits an update. A Flask API (api.py) additionally exposes the same forecast data programmatically through GET /, GET /api/forecast, and GET /api/forecast/<city> endpoints.

11. Known Limitations

- Hopsworks' free-tier serverless job execution queue can introduce two to four minutes of latency per feature store materialization job, since jobs run on a shared compute slot rather than dedicated infrastructure.
- The Streamlit Community Cloud free tier throttles CPU allocation under platform-wide load and puts inactive applications to sleep after extended periods without visitors; this affects only the live dashboard's momentary responsiveness and does not affect the underlying automated pipelines, which continue running on schedule regardless of dashboard traffic.
- A deep learning model (for example, a TensorFlow-based forecaster) was considered as part of the broader model comparison scope described in the original specification, but was not required in production once XGBoost demonstrated consistently superior performance across all three horizons against both baseline model families on this dataset.
- Forecast accuracy is fundamentally bounded by the accuracy of Open-Meteo's underlying weather and air-quality forecasts at the 24h/48h/72h horizons, since the system's own models depend on those forecasts as input features rather than generating independent atmospheric predictions.

12. Conclusion

Pearls AQI Predictor satisfies every core requirement of the original project specification: automated feature collection and engineering, a live feature store, multi-model training and evaluation spanning statistical, ensemble, gradient-boosted, and deep-learning approaches with standard regression metrics, automated daily retraining and hourly forecasting through CI/CD, leakage-safe multi-horizon forecasting, SHAP-based explainability, and a public-facing dashboard with hazard alerting.

The most valuable outcome of this project was not the final model choice but the process of discovering and correcting a genuine data leakage problem: an early baseline's implausibly high accuracy ($R^2 \approx 0.99$, achieved with a random train-test split) was recognized as a warning sign rather than a success, traced to its root cause, and used to redesign the entire feature and evaluation pipeline around strict forecast-time information availability. The resulting leakage-safe XGBoost models, at R^2 of 0.877/0.822/0.806 across the three horizons, represent honest, defensible performance rather than inflated validation metrics that would not survive contact with real production use.

Beyond model development, the system was hardened through iterative debugging of real production issues — schema type mismatches, missing streaming dependencies, job queue contention, race conditions between automated and manual git commits, and deployment dependency isolation — that are representative of the operational challenges in deploying serverless ML pipelines against a managed feature store, rather than issues that would surface in a purely notebook-based or offline implementation. The result is a system that continues to ingest data, retrain, and forecast without manual intervention, with model quality and system health both independently verifiable through the Hopsworks console and the GitHub Actions run history.